OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Database Concepts & Relational Algebra

File Systems vs DBMS, Relational Model, Keys Hierarchy, Algebraic Operators & Integrity Rules

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B1: Topic 5 (Class 12)	Database Concepts: Introduction to database concepts, limitations of traditional file processing systems, DBMS advantages, 3-level ANSI-SPARC architecture, data independence.	Identifying file system flaws (redundancy, anomaly, isolation), difference between logical and physical data independence, ACID properties.
Section B1: Topic 5 (Class 12)	Relational Data Model: Relation, Tuple, Attribute, Cardinality, Degree, Domain, Keys (Candidate, Primary, Alternate, Foreign), Integrity constraints.	Degree (columns) vs Cardinality (rows), Foreign Key NULL value rule, Entity Integrity vs Referential Integrity vs Domain Integrity.
Section B1: Topic 5 (Class 12)	Relational Algebra: Selection (σ), Projection (π), Union ($\cup$), Set Difference ($-$), Cartesian Product ($\times$), Intersection ($\cap$), Natural Join ($\bowtie$).	Union compatibility preconditions, Degree and Cardinality calculations for Cartesian Product ($\times$) and Natural Join ($\bowtie$), duplicate tuple handling.

EXAM PATTERN

CUET Examination Directives: Direct numerical questions regularly ask for the Degree and Cardinality of a Cartesian Product ($R \times S$) or Natural Join ($R \bowtie S$). Conceptual questions heavily test Foreign Key constraints (whether FK can be NULL), Candidate Key vs Alternate Key definitions, and the difference between Selection (σ horizontal slice) and Projection (π vertical slice with duplicate elimination).

Database Concepts & DBMS Architecture

Before modern database management systems, data was stored in decentralized operating system files managed by distinct application programs. This **Traditional File Processing System** suffered from critical structural vulnerabilities that led directly to the development of modern Database Management Systems (DBMS):

Limitations of Traditional File Processing Systems

1. **Data Redundancy:** Identical data elements are stored redundantly in multiple independent files (e.g., student address stored separately in Accounts, Library, and Examination files), causing excessive storage waste.
2. **Data Inconsistency:** Multiple copies of the same data get updated asynchronously. An address change updated in the Examination department leaves the Library file outdated, producing conflicting records.
3. **Data Isolation & Difficult Data Access:** Storing files in varied formats makes writing new ad-hoc reporting programs arduous and time-consuming.
4. **Concurrent Access Anomalies:** When multiple users read and write to the same flat file simultaneously, file-locking issues arise, leading to race conditions and corrupted records.
5. **Integrity & Security Problems:** Data constraints (e.g., balance ≥ 0) must be hardcoded into every individual application program rather than being enforced centrally at the data tier.
6. **Lack of Atomicity (Crash Recovery):** Flat files do not guarantee all-or-nothing transactions. If power fails mid-transfer during a banking operation, funds are debited from one account without reaching the recipient.

Advantages of Database Management Systems (DBMS)

A **DBMS** is a specialized software suite designed to define, construct, manipulate, query, and maintain databases with centralized governance:

- **Controlled Redundancy:** Data is stored once and shared across multiple application views.
- **Data Integrity:** Enforces declarative constraints centrally (e.g., Primary Key, Foreign Key, Domain checks).
- **ACID Transactional Guarantees:** Ensures **Atomicity** (all-or-nothing execution), **Consistency** (valid state transitions), **Isolation** (concurrent transactions do not interfere), and **Durability** (committed changes survive system crashes).

The Three-Level ANSI-SPARC Database Architecture

To isolate user applications from physical storage details, databases implement a 3-tier abstraction model:

1. **Internal / Physical Level:** The lowest level describing **how** data is physically stored on disk (block allocations, B-tree indexes, hashing buckets, data compression, encryption).
2. **Conceptual / Logical Level:** The middle level describing **what** data is stored in the database and the relationships among entities (table definitions, attribute data types, integrity constraints). Managed by Database Administrators (DBAs).
3. **External / View Level:** The highest level describing tailored end-user views. Different users see different subsets of data (e.g., a cashier sees account balances but not customer passwords).

Data Independence

Data independence represents the capacity to modify a schema definition at one abstraction level without altering schema definitions at the adjacent higher level:

- **Physical Data Independence:** The ability to alter the physical storage structure or access methods (e.g., switching from HDD to SSD, rebuilding B-tree indexes, changing storage encoding) without altering the conceptual/logical schema or application code. Physical schema tuning does not break application queries.
- **Logical Data Independence:** The capacity to alter the conceptual schema (e.g., adding a new attribute, splitting an entity into two normalized relations, or modifying relationship cardinalities) without altering external views or breaking existing user applications. Logical data independence is substantially more challenging to achieve than physical data independence because application logic is inherently bound to logical data structures.

Database Users & Actors on the Scene

In database administration and enterprise deployments, distinct classes of actors interact with the DBMS:

- **Database Administrators (DBA):** Holds superuser authority over the entire database environment. Responsibilities include defining the conceptual and physical schemas, configuring access controls and security authorizations, monitoring system performance, scheduling periodic backups, and orchestrating crash recovery protocols.
- **Database Designers:** Responsible for identifying business requirements, discovering entities and relationships, constructing Entity-Relationship (ER) models, defining relational schemas, and establishing integrity constraints prior to database implementation.
- **Application Programmers:** Software developers who write database client applications (using Python, Java, C++, SQL APIs) with embedded Data Manipulation Language (DML) queries to present data to business workflows.
- **End Users:** Individuals whose daily job functions require accessing database contents:
  - **Casual / Ad-hoc Users:** Access data intermittently by issuing sophisticated ad-hoc analytical queries using interactive SQL interfaces or reporting dashboards.

- **Naive / Parametric Users:** Constitute the vast majority of database users (e.g., bank tellers, railway ticket reservation clerks, e-commerce shoppers). They execute pre-compiled canned transactions with fixed parameters via simple graphical user interfaces.
- **Sophisticated Users:** Engineers, data scientists, and analysts who thoroughly comprehend DBMS capabilities and write custom statistical or machine learning pipelines interacting directly with the engine.
- **Standalone Users:** Maintain personal databases using off-the-shelf software packages (e.g., personal tax or inventory trackers).

Comparison of Data Models: Relational vs Hierarchical vs Network

Before the relational paradigm became dominant, data systems utilized hierarchical and network architectures. Understanding these distinctions is a regular focus of CUET conceptual questions:

Evaluation Dimension	Relational Model	Hierarchical Model	Network Model
Underlying Data Structure	Two-dimensional tables (relations) with rows and columns.	Hierarchical tree structure with parent-child segment nodes.	Arbitrary directed graph structure with record types and links.
Relationship Cardinalities	Naturally supports 1:1, 1:N, and M:N relationships via foreign keys and junction tables.	Strictly restricted to 1:1 and 1:N parent-child relationships; cannot model M:N directly.	Supports 1:1, 1:N, and complex M:N relationships directly using member sets.
Parent/Child Constraints	Independent entities; child can exist without parent (unless constrained by foreign key).	Every child node must have exactly one parent node (except the unique root node).	A child (member) record can have multiple parent (owner) records.
Data Access & Navigation	Declarative: User specifies **what** data is needed (non-procedural SQL); engine optimizes path.	Procedural: Navigation via explicit pointer traversal through tree branches.	Procedural: Navigation via complex circular pointer chains and linked sets.
Physical Independence	Very High: Physical storage changes do not impact logical queries.	Low: Application logic is tightly coupled to physical pointer trees.	Low: Pointer modifications necessitate widespread application rewrites.

The Relational Data Model

Proposed by Dr. E.F. Codd of IBM in 1970, the **Relational Model** represents all data and relationships exclusively through two-dimensional tabular mathematical structures termed **relations**:

Formal Relational Terminology

Formal Model Term	Informal Table Analogy	Formal Mathematical Definition & Constraints
Relation	Table	A named two-dimensional table representing an entity set or relationship set.
Tuple	Row / Record	A single horizontal record within a relation representing a specific entity instance.
Attribute	Column / Field	A named vertical column representing a specific property or characteristic.
Domain	Data Type / Permitted Values	The set of all permissible atomic values that an attribute can legally assume.
Degree	Number of Columns	The total count of attributes comprising the relation schema. Represented as n.
Cardinality	Number of Rows	The total count of tuples currently stored in the relation instance. Represented as m.

Mathematical Formulation of a Relation

Given n domains $D_1, D_2, ..., D_n$, a relation R is mathematically defined as a subset of the Cartesian product of these domains:

$$R \subseteq D_1 \times D_2 \times ... \times D_n$$

A tuple $t \in R$ is an ordered list of n atomic values: $t = (v_1, v_2, ..., v_n)$ where $v_i \in D_i$.

Fundamental Invariant Properties of Relations

1. **Tuple Uniqueness:** No two tuples in a relation can be identical duplicates (enforced by candidate keys).

2. **Unordered Tuples:** The physical sequence of rows is completely irrelevant to the database.
3. **Unordered Attributes:** The physical order of columns is completely irrelevant to the relational schema.
4. **Atomic Values (First Normal Form - 1NF):** Every attribute value in every tuple must be strictly atomic (indivisible). Multi-valued sets or composite arrays within a single attribute cell are prohibited in relational theory.

Relational Data Model & Keys Architecture

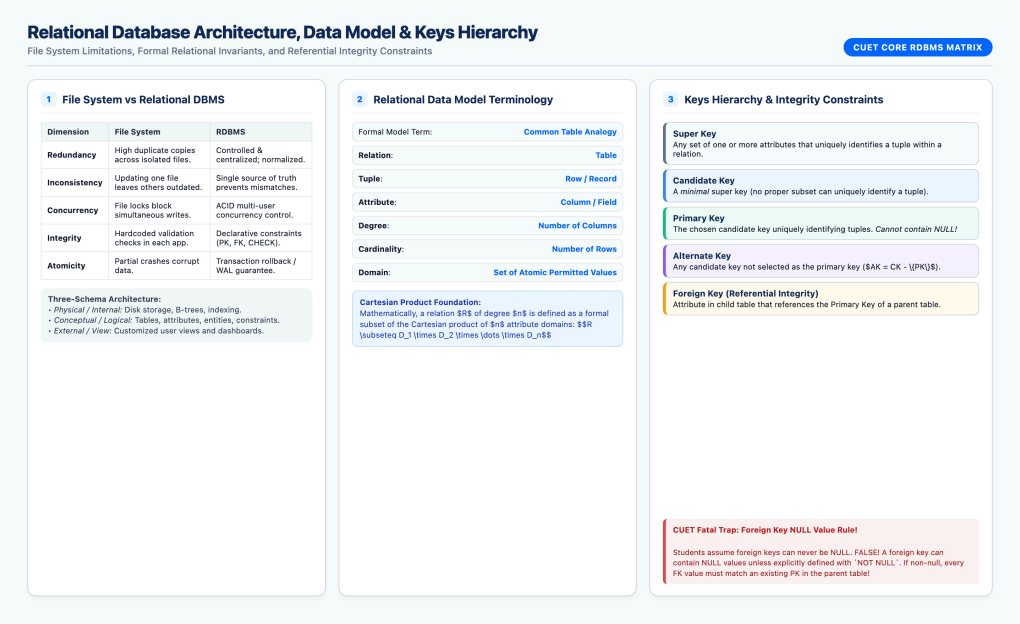


Figure 1: Relational Architecture Matrix: File System vs DBMS, Model Terminology, and Keys Hierarchy

The Relational Keys Hierarchy

Keys are attribute combinations that enforce tuple uniqueness and model relationships across entities:

1. **Super Key:** Any set of one or more attributes that collectively and uniquely identifies a tuple within a relation. A super key may contain extraneous or redundant attributes.
2. **Candidate Key:** A **minimal super key**. A set of attributes that uniquely identifies tuples such that no proper subset can uniquely identify them. A relation may possess multiple candidate keys.
3. **Primary Key:** The specific candidate key chosen by the database designer as the principal mechanism for identifying tuples uniquely within the relation.
   - **Entity Integrity Constraint:** A Primary Key **CANNOT be NULL**! Every tuple must have a distinct, non-null primary key value.
4. **Alternate Key:** Any candidate key that was **not** selected as the primary key:
$$\text{Alternate Keys} = \text{Candidate Keys} - \{\text{Primary Key}\}$$
5. **Foreign Key:** An attribute (or combination of attributes) in a relation (child table) whose values must match the Primary Key of another relation (parent table), or be strictly NULL.
   - **Referential Integrity Constraint:** Enforces that child records cannot reference non-existent parent entities.

Relational Integrity Constraints & Violation Actions

To maintain database consistency, DBMS engines enforce three foundational classes of integrity rules:

- **1. Entity Integrity:** The primary key of a base relation cannot accept NULL values or duplicates.
- **2. Domain Integrity:** Attribute values must belong to the specified data type and satisfy domain ranges (e.g. `Age >= 0`).
- **3. Referential Integrity Violation Actions:** When a user attempts to `DELETE` a parent tuple or `UPDATE` a parent primary key that is referenced by child foreign keys, the DBMS can enforce one of four declarative actions:
  - `CASCADE`: Automatically propagates the delete or update to all referencing child tuples.
  - `SET NULL`: Sets the foreign key column in all matching child tuples to `NULL`.
  - `RESTRICT` / `NO ACTION`: Strictly rejects the parent operation if matching child records exist.
  - `SET DEFAULT`: Sets the foreign key column in all matching child tuples to a designated default value.

KEYS & REFERENTIAL INTEGRITY ANALYSIS

Problem: Consider the relations:

- `DEPARTMENT(DeptId, DeptName, Location)` where `DeptId` is Primary Key.

- `STUDENT(AdmNo, RollNo, Name, Email, DeptId)` containing 50 student records.

Both `AdmNo` and `Email` are unique.

1. What is the Degree and Cardinality of `STUDENT`?
2. Identify the Candidate Keys, Primary Key, and Alternate Keys.
3. If `DeptId` in `STUDENT` references `DEPARTMENT(DeptId)` with `ON DELETE SET NULL`, what happens if Department `10` is deleted?

Solution:

- **Part 1:** Degree = 5 (5 attributes: AdmNo, RollNo, Name, Email, DeptId). Cardinality = 50 (50 tuples).
- **Part 2:**
  - Candidate Keys: `AdmNo` and `Email` (both minimally unique).
  - If `AdmNo` is chosen as the Primary Key, then `Email` is the Alternate Key.
- **Part 3:** `DeptId` is a **Foreign Key**. If department `10` is deleted from `DEPARTMENT`, all students previously enrolled in department `10` will have their `DeptId` field updated to `NULL`, maintaining referential integrity without deleting student records.

Candidate Key Determination & Super Key Combinatorics

Determining candidate keys and counting super keys are fundamental analytical skills rigorously tested in the CUET Computer Science examination:

1. Functional Dependency ($X \to Y$)

A functional dependency $X \to Y$ asserts that if two tuples agree on the values of attribute set X, they must also agree on attribute set Y. X is termed the **determinant**, and Y is the **dependent**.

2. Attribute Set Closure (X^+)

The closure X^+ with respect to a set of functional dependencies F is the complete set of all attributes functionally determined by X:

1. Initialize $X^+ = X$.
2. For each FD $Y \to Z \in F$, if $Y \subseteq X^+$, then update $X^+ = X^+ \cup Z$.
3. Repeat step 2 until no further attributes can be added.
4. If X^+ equals the entire relation schema R, then X is a **Super Key**! If no proper subset of X determines R, then X is a **Candidate Key**.

3. Combinatorics of Super Keys

If a relation R consists of n attributes:

- **Single Attribute Candidate Key:** If attribute A is the sole candidate key, any subset of the remaining $n-1$ attributes can be combined with A to form a super key. Thus, the total number of super keys is:
$$\text{Total Super Keys} = 2^{n-1}$$
- **Two Attribute Candidate Keys (A and B):** By the Principle of Inclusion-Exclusion, the number of super keys containing A, B, or both is:
$$\text{Total Super Keys} = |S_A| + |S_B| - |S_{AB}| = 2^{n-1} + 2^{n-1} - 2^{n-2}$$
- **Composite Candidate Key (AB):** If the minimal candidate key requires two attributes together (AB), the remaining $n-2$ attributes can freely combine with AB:
$$\text{Total Super Keys} = 2^{n-2}$$

FINDING CANDIDATE KEYS AND SUPER KEYS

Problem: Consider the relation schema $R(A, B, C, D)$ with functional dependencies:
$$F = \{A \to B, B \to C, C \to D\}$$

1. Compute the closure A^+, B^+, and C^+.
2. Determine all Candidate Keys of R.
3. Calculate the total number of Super Keys for R.

Solution:

- **Part 1:**
  - $A^+ = \{A\} \to \{A, B\} \to \{A, B, C\} \to \{A, B, C, D\} = R$. (Since A^+ contains all attributes, A is a candidate key).
  - $B^+ = \{B\} \to \{B, C\} \to \{B, C, D\} \neq R$ (cannot determine A).
  - $C^+ = \{C\} \to \{C, D\} \neq R$ (cannot determine A or B).
- **Part 2:**
  - Notice that attribute A does not appear on the right-hand side of any FD in F. Therefore, A must be present in **every** candidate key!
  - Since A alone determines all attributes ($A^+ = R$), the set $\{A\}$ is minimal.
  - Thus, R has exactly **one Candidate Key**: $\{A\}$.
- **Part 3:**
  - Relation R has $n = 4$ attributes (A, B, C, D).

- Any super key must contain the candidate key A. The remaining 3 attributes (B, C, D) can either be included or excluded independently (2^3).
- Total Super Keys $= 2^{4-1} = 2^3 = 8$.
- The 8 super keys are: $\{A\}$, $\{A, B\}$, $\{A, C\}$, $\{A, D\}$, $\{A, B, C\}$, $\{A, B, D\}$, $\{A, C, D\}$, and $\{A, B, C, D\}$.

Relational Algebra: Unary Operators

Relational Algebra is a theoretical procedural query language introduced by E.F. Codd. It consists of a collection of algebraic operations that take one or two relations as input and produce a **new relation** as their output, enabling mathematical query composition:

1. Selection Operator (σ)

The Selection operator performs a **horizontal filtering** of a relation. It extracts only those tuples that satisfy a specified boolean condition (predicate) C:

$$\sigma_C(R) = \{t \texttt{ mid } t \in R \text{ and } C(t) = \text{True}\}$$

- **Predicate Operators:** Comparison operators $(=, \neq, <, \leq, >, \geq)$ combined using logical connectives $(\wedge, \vee, \neg)$.
- **Degree Invariant:** The degree of the resulting relation is **strictly identical** to the input relation:
$$\text{Degree}\left(\sigma_{C(R)}\right) = \text{Degree}(R)$$
- **Cardinality Invariant:** The cardinality is at most equal to the input cardinality:
$$0 \leq \text{Cardinality}\left(\sigma_{C(R)}\right) \leq \text{Cardinality}(R)$$

2. Projection Operator (π)

The Projection operator performs a **vertical filtering** of a relation. It selects specified attributes and extracts only those columns, discarding the rest:

$$\pi_{A_1, A_2, \ldots, A_k}(R)$$

- **Degree Invariant:** The degree equals the number of attributes specified in the projection list:
$$\text{Degree}\left(\pi_{A_1, \ldots, A_k}(R)\right) = k$$
- **Duplicate Elimination Rule:** By definition, a relation is a mathematical set of distinct tuples. Therefore, the Projection operator **strictly eliminates duplicate rows** from the output relation!
- **Cardinality Invariant:** Because duplicate rows are purged, the cardinality of a projection may be strictly smaller than the input:
$$\text{Cardinality}\left(\pi_{L(R)}\right) \leq \text{Cardinality}(R)$$

3. Rename Operator (ρ)

Allows renaming a relation or its attributes without altering its data or cardinality:

- $\rho_{S(R)}$: Renames relation R to S.
- $\rho_{S(B_1, \ldots, B_n)}(R)$: Renames relation R to S and renames attributes to $B_1, \ldots, B_n$.

EVALUATING SELECTION AND PROJECTION EXPRESSIONS

Problem: Consider the relation `EMPLOYEE`:

- E1: (101, 'Alice', 'Sales', 50000)
- E2: (102, 'Bob', 'IT', 65000)
- E3: (103, 'Charlie', 'Sales', 50000)
- E4: (104, 'David', 'IT', 70000)

Evaluate the degree and cardinality of:

1. $\sigma_{\text{Dept = 'Sales' and Salary > 40000}}(\text{EMPLOYEE})$
2. $\pi_{\text{Dept, Salary}}(\text{EMPLOYEE})$

Solution:

- **Part 1 (Selection):** Tuples satisfying condition: E1 ('Alice', 'Sales', 50000) and E3 ('Charlie', 'Sales', 50000). Both tuples match.
  - Result has Degree = 4 (all original columns retained) and Cardinality = 2.
- **Part 2 (Projection):** Extracting columns `(Dept, Salary)`:
  - E1 produces `('Sales', 50000)`
  - E2 produces `('IT', 65000)`
  - E3 produces `('Sales', 50000)` (Duplicate of E1!)

- E4 produces `('IT', 70000)`
- Applying duplicate elimination leaves 3 distinct tuples: `('Sales', 50000)`, `('IT', 65000)`, `('IT', 70000)`.
- Result has Degree = 2 and Cardinality = 3 (strictly < 4 due to duplicate removal).

Relational Algebra Operators Lifecycle

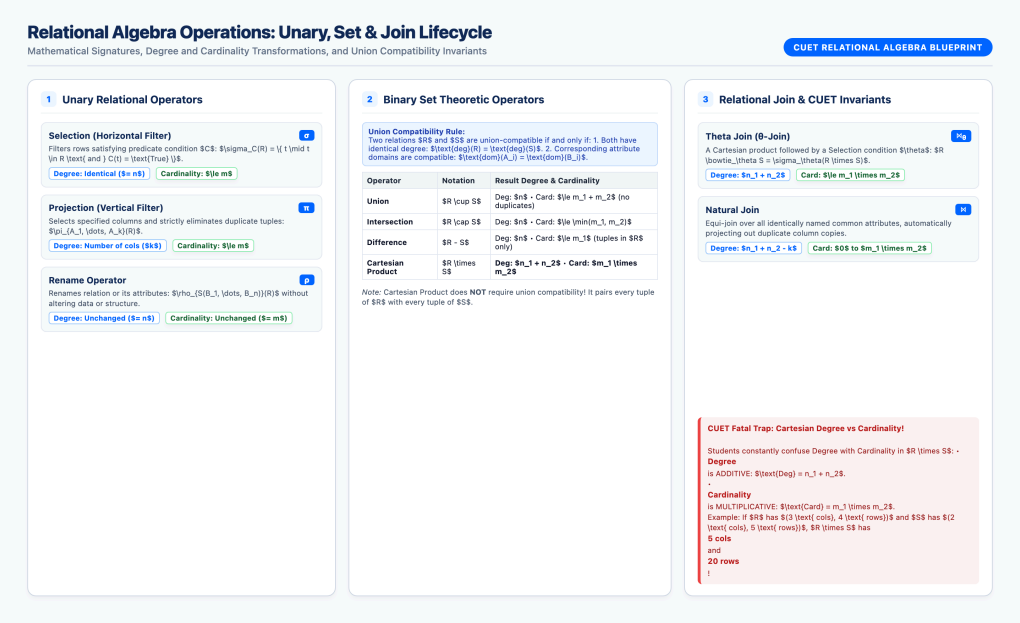


Figure 2: Relational Algebra Operator Invariants: Unary Filtering, Set Preconditions, and Join Lifecycles

Union Compatibility (Type Compatibility)

Before binary set operators (Union, Intersection, Set Difference) can be applied between two relations R and S, they must satisfy the strict condition of **Union Compatibility**:

1. Both relations must have the **identical degree**:
$$\text{Degree}(R) = \text{Degree}(S) = n$$
2. The domain of the i-th attribute of R must be **compatible** with the domain of the i-th attribute of S for all $i \in \{1, 2, ..., n\}$:
$$\text{dom}(A_i) = \text{dom}(B_i)$$

Binary Set Theoretic Operators

Operator	Mathematical Notation	Degree & Cardinality Guarantees
Union	$R \cup S$	Requires union compatibility. Combines tuples from both relations without duplicates. Degree $= n$. Cardinality: $\max(m_1, m_2) \le \text{Card} \le m_1 + m_2$.
Intersection	$R \cap S$	Requires union compatibility. Retains only tuples present in both R and S. Degree $= n$. Cardinality: $0 \le \text{Card} \le \min(m_1, m_2)$.
Set Difference	$R - S$	Requires union compatibility. Retains tuples belonging to R that do not appear in S. Degree $= n$. Cardinality: $0 \le \text{Card} \le m_1$. Note: $R - S \neq S - R$ (Non-commutative).
Cartesian Product	$R \times S$	Does **NOT** require union compatibility! Pairs every tuple in R with every tuple in S. Degree: $n_1 + n_2$ (Additive). Cardinality: $m_1 \times m_2$ (Multiplicative).

Cartesian Product & Cross-Product Operations

Formal Definition & Characteristics

The Cartesian Product (or Cross Product) $R \times S$ combines two relations R and S of arbitrary degrees and schemas into a single composite relation:

$$R \times S = \{t_1 t_2 \texttt{ mid } t_1 \in R \text{ and } t_2 \in S\}$$

where $t_1 t_2$ represents the concatenation of tuples t_1 and t_2.

The Golden Rules of Cartesian Product

1. **Degree is Strictly Additive:**
$$\text{Degree}(R \times S) = \text{Degree}(R) + \text{Degree}(S) = n_1 + n_2$$
2. **Cardinality is Strictly Multiplicative:**
$$\text{Cardinality}(R \times S) = \text{Cardinality}(R) \times \text{Cardinality}(S) = m_1 \times m_2$$
3. **Commutative Property:** While the column order changes, $R \times S$ and $S \times R$ contain identical information.

CARTESIAN PRODUCT DEGREE AND CARDINALITY CALCULATIONS

Problem: Relation R has 4 attributes and 10 tuples. Relation S has 3 attributes and 5 tuples.

1. What is the Degree and Cardinality of $R \times S$?
2. If an empty relation T (Cardinality $= 0$, Degree $= 2$) is cross-multiplied with R, what is the result?

Solution:

- **Part 1:**
  - $\text{Degree}(R \times S) = \text{Degree}(R) + \text{Degree}(S) = 4 + 3 = 7$.
  - $\text{Cardinality}(R \times S) = \text{Cardinality}(R) \times \text{Cardinality}(S) = 10 \times 5 = 50$.
- **Part 2:**
  - $\text{Degree}(R \times T) = 4 + 2 = 6$.
  - $\text{Cardinality}(R \times T) = 10 \times 0 = 0$ (an empty relation).

Relational Join Operations

A **Join** operator combines related tuples from two relations into single tuples based on a join condition. It represents the foundation of relational query execution:

1. Theta Join ($\bowtie_\theta$)

A Theta Join is formally defined as a Cartesian Product followed immediately by a Selection condition:

$$R \underset{\theta}{\bowtie} S = \sigma_\theta(R \times S)$$

- Degree: $\text{Degree}\left(R \underset{\theta}{\bowtie} S\right) = n_1 + n_2$.
- Cardinality: Ranges from 0 (if no tuples satisfy θ) to $m_1 \times m_2$.

2. Equi-Join

A specific case of Theta Join where the join condition θ uses exclusively the equality operator ($=$):

$$R \underset{R.A=S.B}{\bowtie} S$$

Both attributes $R.A$ and $S.B$ are retained in the resulting schema, producing duplicate column values.

3. Natural Join ($\bowtie$)

The **Natural Join** is an Equi-Join performed over all identically named common attributes, with the critical distinction that **duplicate common attribute columns are automatically projected out**:

- **Degree Formula for Natural Join:**
$$\text{Degree}(R \bowtie S) = \text{Degree}(R) + \text{Degree}(S) - k$$
where k represents the number of common attribute names shared between R and S.
- **Cardinality Bounds:**
$$0 \leq \text{Cardinality}(R \bowtie S) \leq \text{Cardinality}(R) \times \text{Cardinality}(S)$$
If the join attribute is a Primary Key in S and a Foreign Key in R, then $\text{Cardinality}(R \bowtie S) \leq \text{Cardinality}(R)$.

NATURAL JOIN EXECUTION TRACE

Problem: Consider relations $R(A, B)$ and $S(B, C)$:

- $R = \{(1, 10), (2, 20), (3, 30)\}$
- $S = \{(10, \text{X}), (20, \text{Y}), (40, \text{Z})\}$

Compute the resulting schema, degree, and tuples of $R \bowtie S$.
Solution:

- Common attribute is B ($k = 1$).
- Result schema: (A, B, C). (Attribute B is retained once, not duplicated).
- Degree $= 2 + 2 - 1 = 3$.
- Matching on B:
  - Tuple $(1, 10)$ in R matches $(10, \text{X})$ in $S \rightarrow (1, 10, \text{X})$.
  - Tuple $(2, 20)$ in R matches $(20, \text{Y})$ in $S \rightarrow (2, 20, \text{Y})$.
  - Tuple $(3, 30)$ in R has no match in S ($B = 30$ not in S).
  - Tuple $(40, \text{Z})$ in S has no match in R ($B = 40$ not in R).
- **Final Result of $R \bowtie S$:** $\{(1, 10, \text{X}), (2, 20, \text{Y})\}$.
- Cardinality $= 2$.

4. Outer Join Operations ($\ltimes$, $\bowtie$, $\rtimes$)

In an standard (inner) Natural Join or Equi-Join, tuples that fail to find a matching partner in the opposing relation are silently discarded (known as **dangling tuples**). **Outer Joins** preserve these dangling tuples by populating unmatched attributes with `NULL` values:

Left Outer Join ($\ltimes$)

Preserves **all** tuples from the left relation R, regardless of whether a matching tuple exists in S. For tuples in R having no counterpart in S, all attributes contributed by S are set to `NULL`:

- Degree: $\text{Degree}(R \ltimes S) = \text{Degree}(R \bowtie S)$.
- Cardinality Invariant: Every tuple in R appears at least once:

$$\text{Cardinality}(R) \leq \text{Cardinality}(R \ltimes S) \leq \text{Cardinality}(R) \times \text{Cardinality}(S)$$

Right Outer Join ($\rtimes$)

Preserves **all** tuples from the right relation S. For tuples in S with no matching tuple in R, the attributes of R are filled with `NULL`:

- Degree: $\text{Degree}(R \rtimes S) = \text{Degree}(R \bowtie S)$.
- Cardinality Invariant: Every tuple in S appears at least once:

$$\text{Cardinality}(S) \leq \text{Cardinality}(R \rtimes S) \leq \text{Cardinality}(R) \times \text{Cardinality}(S)$$

Full Outer Join ($\bowtie$)

Preserves all tuples from **both** relations R and S. Matches are concatenated as normal; unmatched tuples from either side are retained and padded with `NULL`s for the missing attributes:

$$R \bowtie S = (R \ltimes S) \cup (R \rtimes S)$$

- Cardinality Invariant:

$$\max(\text{Card}(R), \text{Card}(S)) \leq \text{Cardinality}(R \bowtie S) \leq \text{Card}(R) \times \text{Card}(S)$$

OUTER JOINS ON DANGLING TUPLES

Problem: For $R(A, B) = \{(1, 10), (2, 20), (3, 30)\}$ and $S(B, C) = \{(10, \text{X}), (20, \text{Y}), (40, \text{Z})\}$, compute:

1. $R \ltimes S$ (Left Outer Join): Preserves all tuples of R. Unmatched $(3, 30)$ gets $C = \text{NULL} \rightarrow$ $\{(1, 10, \text{X}), (2, 20, \text{Y}), (3, 30, \text{NULL})\}$ (Cardinality $= 3$).
2. $R \rtimes S$ (Right Outer Join): Preserves all tuples of S. Unmatched $(40, \text{Z})$ gets $A = \text{NULL} \rightarrow$ $\{(1, 10, \text{X}), (2, 20, \text{Y}), (\text{NULL}, 40, \text{Z})\}$ (Cardinality $= 3$).
3. $R \bowtie S$ (Full Outer Join): Preserves unmatched from both sides $\rightarrow \{(1, 10, \text{X}), (2, 20, \text{Y}), (3, 30, \text{NULL}), (\text{NULL}, 40, \text{Z})\}$ (Cardinality $= 4$).

Advanced Relational Operations & Query Composition

The Division Operator ($\div$)

The Division operator ($R \div S$) is utilized for queries containing the phrase **"for all"** or **"for every"**. Given relation R of degree n and relation S of degree m (where all attributes of S are a proper subset of R, and $n > m$):

$$\text{Degree}(R \div S) = \text{Degree}(R) - \text{Degree}(S) = n - m$$

A tuple t appears in $R \div S$ if and only if for every tuple $s \in S$, the concatenated tuple ts exists in R.

- **Formal Algebraic Definition:**

$$R \div S = \pi_{R-S}(R) - \pi_{R-S}((\pi_{R-S}(R) \times S) - R)$$

- **Classic Example:** "Find students who have enrolled in **all** courses offered by the Computer Science department."

STEP-BY-STEP EVALUATION OF RELATIONAL DIVISION

Problem: Let relation `ENROLL(Student, Course)` contain:

- `(Alice, CS101)`, `(Alice, CS102)`, `(Alice, MATH)`
- `(Bob, CS101)`
- `(Charlie, CS101)`, `(Charlie, CS102)`

Let relation `REQUIRED(Course)` contain `{(CS101), (CS102)}`. Evaluate `ENROLL` ÷ `REQUIRED`.
Solution:

1. Target schema: Attributes in `ENROLL` not in `REQUIRED` = {Student}. Degree = 2 − 1 = 1.
2. For each unique student, check if they are paired with **every** course in `REQUIRED`:
   - **Alice:** Has pairs `(Alice, CS101)` and `(Alice, CS102)`. Both required courses present! → **Alice qualifies**.
   - **Bob:** Has `(Bob, CS101)`, but lacks `(Bob, CS102)`. Missing course! → **Bob excluded**.
   - **Charlie:** Has `(Charlie, CS101)` and `(Charlie, CS102)`. Both required courses present! → **Charlie qualifies**.
3. **Result Relation:** `{(Alice), (Charlie)}`. Degree = 1, Cardinality = 2.

Classic Relational Query Formulations

A frequent exam question in CUET is expressing analytical queries using pure relational algebra without aggregate functions:

1. **Finding the Highest Salary without MAX():**
   - Step 1: Form the Cartesian product of `EMP` with itself to find all salaries that are strictly less than some other salary:
   $$\text{LowerSalaries} = \pi_{\text{E1.Salary}}\left(\sigma_{\text{E1.Salary} < \text{E2.Salary}}(\rho_{\text{E1}}(\text{EMP}) \times \rho_{\text{E2}}(\text{EMP}))\right)$$
   - Step 2: Subtract these lower salaries from the complete set of all salaries using Set Difference:
   $$\text{MaxSalary} = \pi_{\text{Salary}}(\text{EMP}) - \text{LowerSalaries}$$
   - The result is a single-column relation containing exclusively the maximum salary!
2. **Finding Elements in A but NOT in B using Left Outer Join:** In SQL and relational systems, Set Difference can be expressed via Outer Join:
   $$\pi_{R.\text{ID}}(\sigma_{S.\text{ID is NULL}}(R \bowtie S))$$

Relational Query Synthesis & Equivalence Laws

Relational DBMS query optimizers transform user queries into mathematically equivalent expressions that execute with drastically fewer disk I/O operations:

1. **Pushing Selections Down:** Performing selection σ **before** Cartesian product or join drastically reduces intermediate table sizes:
   $$\sigma_C(R \bowtie S) \equiv \left(\sigma_{C(R)}\right) \bowtie S \quad (\text{if } C \text{ involves only attributes of } R)$$
2. **Pushing Projections Down:** Eliminating unneeded attributes prior to computing joins reduces tuple width and memory footprint:
   $$\pi_L(R \bowtie S) \equiv \pi_L\left(\pi_{L_1}(R) \bowtie \pi_{L_2}(S)\right)$$
3. **Commutativity and Associativity of Joins:**
   - $R \bowtie S \equiv S \bowtie R$ (Commutative)
   - $(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$ (Associative)

   Query engines reorder joins so that relations with smaller cardinalities are joined first, minimizing memory buffers.

QUERY OPTIMIZATION VIA SELECTION PUSHDOWN

Problem: Compare costs for $R(A, B)$ (1000 tuples) and $S(B, C)$ (2000 tuples) with 5 tuples having $A = 50$:

- Plan 1: $\sigma_{\text{A = 50}}(R \bowtie S)$ vs Plan 2: $(\sigma_{\text{A = 50}}(R)) \bowtie S$.

Solution: Plan 1 computes the entire join $R \bowtie S$ first (generating thousands of tuples) before filtering. Plan 2 pushes selection down, filtering R to 5 tuples **before** joining with S. Pushing selections down reduces intermediate join memory and I/O by orders of magnitude, serving as the core heuristic in relational optimizers.

Master Synthesis Mindmap: Database Concepts & Relational Algebra

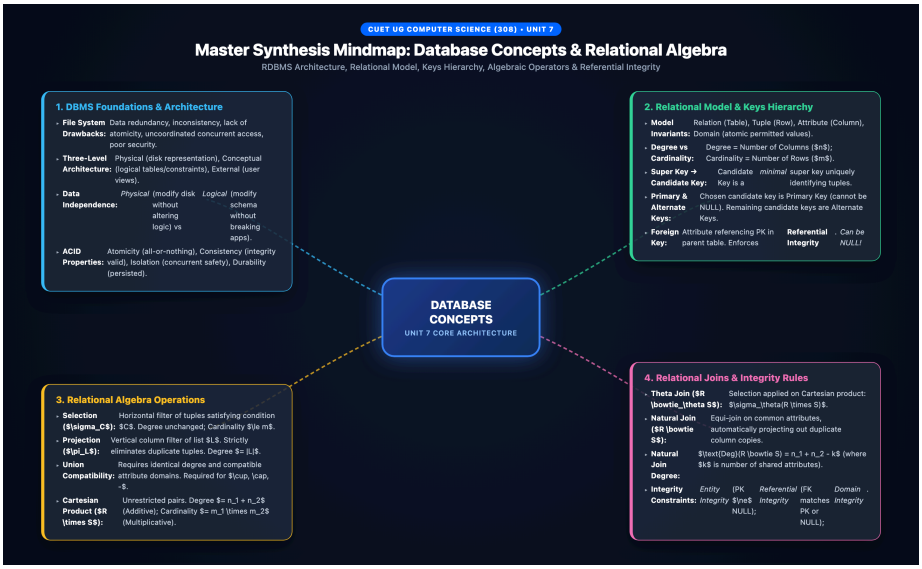


Figure 3: Master Synthesis Mindmap: Database Concepts, Relational Data Model, Keys & Relational Algebra

OurMocks

Cheatsheet & The Seven Fatal CUET Traps

RAPID REVISION CHEATSHEET: DATABASE CONCEPTS & RELATIONAL ALGEBRA

- **File System Drawbacks:** Redundancy, Inconsistency, Concurrent access conflicts, Data isolation, Lack of Atomicity.
- **Three-Level Architecture:** Physical (storage), Conceptual (logical tables/constraints), External (user views).
- **Data Independence:** Physical (change disk/index without changing logic); Logical (change schema without breaking app code).
- **Relational Model Terms:** Relation (Table), Tuple (Row), Attribute (Column), Domain (Permitted atomic values).
- **Degree vs Cardinality:** Degree $= n$ (Number of columns); Cardinality $= m$ (Number of rows).
- **Keys Hierarchy:** Super Key (any unique set) $\supset$ Candidate Key (minimal unique set) $\rightarrow$ Primary Key (chosen CK, NOT NULL) + Alternate Keys (remaining CKs).
- **Foreign Key:** Attribute referencing Primary Key of parent table. Enforces Referential Integrity. **CAN be NULL**.
- **Integrity Constraints:** Entity Integrity (PK cannot be NULL), Referential Integrity (FK must match PK or be NULL), Domain Integrity (values within valid range/type).
- **Selection (σ):** Horizontal tuple filter. Degree unchanged ($= n$). Cardinality $\leq m$.
- **Projection (π):** Vertical column filter. Strictly removes duplicate rows. Degree $= |L|$. Cardinality $\leq m$.
- **Union Compatibility:** Identical degree AND compatible attribute domains. Required for $\cup, \cap, -$.
- **Cartesian Product ($R \times S$):** Degree $= n_1 + n_2$ (Additive). Cardinality $= m_1 \times m_2$ (Multiplicative).
- **Natural Join ($R \bowtie S$):** Equi-join on common attributes, dropping duplicate column copies. Degree $= n_1 + n_2 - k$.
- **Division ($R \div S$):** Used for "for all" queries. Degree $= n - m$.

THE SEVEN FATAL CUET TRAPS IN DATABASE CONCEPTS & RELATIONAL ALGEBRA

1. **The Foreign Key NULL Trap:** Students assume foreign keys can never contain NULL values. **FALSE!** A foreign key **can** be NULL unless explicitly designated as `NOT NULL`. If it contains a value, that value must exist in the parent table.
2. **Degree vs Cardinality Confusion in Cartesian Product:** Mixing up addition and multiplication: In $R \times S$, **Degree is ADDED** $(n_1 + n_2)$, while **Cardinality is MULTIPLIED** $(m_1 \times m_2)$.
3. **Projection Duplicate Removal Trap:** Assuming Projection preserves the exact number of rows. In pure relational algebra, relations are sets; therefore, Projection **strictly eliminates duplicate rows**, which can reduce cardinality!
4. **Natural Join Duplicate Column Trap:** Assuming Natural Join keeps two copies of the common column. Natural Join automatically projects out one copy of each shared attribute (Degree $= n_1 + n_2 - k$).
5. **Union Compatibility Applicability Trap:** Believing Cartesian Product requires union compatibility. **FALSE!** Cartesian Product works on any two arbitrary relations. Only Union ($\cup$), Intersection ($\cap$), and Difference ($-$) require union compatibility.
6. **Candidate Key vs Super Key:** Every Candidate Key is a Super Key, but not every Super Key is a Candidate Key (a candidate key must be **minimal** with zero redundant attributes).
7. **Set Difference Non-Commutativity:** Writing $R - S = S - R$. Set difference is **non-commutative**! $R - S$ contains tuples in R not in S, whereas $S - R$ contains tuples in S not in R.

CUET Focused Multiple Choice Questions (Part 1)

Q1. Relation R contains 5 attributes and 20 tuples. Relation S contains 3 attributes and 10 tuples. What are the Degree and Cardinality of the resulting relation obtained by evaluating the Cartesian Product $R \times S$?

(A) Degree = 8, Cardinality = 200
(B) Degree = 15, Cardinality = 30
(C) Degree = 8, Cardinality = 30
(D) Degree = 15, Cardinality = 200

Sol. Q1. In a Cartesian Product $R \times S$:

- The Degree is strictly additive: $\text{Degree} = \text{Degree}(R) + \text{Degree}(S) = 5 + 3 = 8$.
- The Cardinality is strictly multiplicative: $\text{Cardinality} = \text{Cardinality}(R) \times \text{Cardinality}(S) = 20 \times 10 = 200$.

Thus, the result has Degree = 8 and Cardinality = 200.

Answer: (A)

Q2. Which of the following statements concerning a **Foreign Key** in a relational database is TRUE according to the rules of Referential Integrity?

(A) A Foreign Key can never contain duplicate values under any circumstances.
(B) A Foreign Key can contain NULL values, provided it has not been defined with a NOT NULL constraint.
(C) A Foreign Key must always reference another column in the exact same table.
(D) A Foreign Key cannot have the same name as the Primary Key it references.

Sol. Q2. Referential integrity requires that if a foreign key contains a non-null value, that value must match an existing primary key value in the referenced parent relation. However, foreign keys are fully permitted to contain NULL values (representing an unassigned relationship) unless explicitly constrained by `NOT NULL`.

Answer: (B)

Q3. A relation `EMPLOYEE(EmpId, Name, Department, Salary)` contains 100 records. There are 5 distinct departments and 20 distinct salary levels. What is the Degree and the maximum possible Cardinality of the relational algebra expression $\pi_{\text{Department}}(\text{EMPLOYEE})$?

(A) Degree = 4, Cardinality = 100
(B) Degree = 1, Cardinality = 100
(C) Degree = 1, Cardinality = 5
(D) Degree = 4, Cardinality = 5

Sol. Q3. The Projection operator $\pi_{\text{Department}}$ projects exactly 1 column (`Department`), so its Degree is strictly 1. Because relational algebra relations are mathematical sets, projection strictly eliminates duplicate tuples. Since there are only 5 distinct departments, the output cardinality is strictly 5.

Answer: (C)

Q4. In the ANSI-SPARC three-level database architecture, what is meant by **Physical Data Independence**?

(A) The ability to modify external views without altering the conceptual schema.
(B) The ability to add new entities to the database without recompiling the operating system.
(C) The ability to change user permissions without altering the database schema.
(D) The capacity to modify the internal/physical storage structures (such as indexes or file organizations) without requiring changes to the conceptual schema.

Sol. Q4. Physical Data Independence refers to the separation between the conceptual logical schema and the physical internal schema. It guarantees that alterations to physical storage (e.g., using new hardware, reorganizing disk blocks, creating B-tree indexes) do not require rewriting conceptual schemas or application programs.

Answer: (D)

Q5. Consider a relation `STUDENT(AdmNo, RollNo, Name, Email, Phone)` where `AdmNo`, `RollNo`, and `Email` are each individually unique for every student. If `AdmNo` is chosen as the Primary Key, what are `RollNo` and `Email` formally termed?

(A) Alternate Keys
(B) Foreign Keys
(C) Composite Keys
(D) Surrogate Keys

Sol. Q5. Any attribute (or minimal set of attributes) that uniquely identifies a tuple is a Candidate Key. Here, `AdmNo`, `RollNo`, and `Email` are all Candidate Keys. The candidate key selected by the DBA is the Primary Key (`AdmNo`). All remaining unselected candidate keys (`RollNo` and `Email`) are formally designated as **Alternate Keys**.

Answer: (A)

CUET Focused Multiple Choice Questions (Part 2)

Q6. Which of the following preconditions MUST be satisfied by two relations R and S before the Union ($R \cup S$) or Set Difference ($R - S$) operations can be performed on them?

(A) Both relations must have the exact same cardinality (number of rows).

(B) Both relations must be Union-Compatible: they must have the same degree, and corresponding attributes must have compatible domains.

(C) Both relations must share at least one foreign key constraint.

(D) Both relations must be stored on the same physical disk drive.

Sol. Q6. Set operations (Union, Intersection, Set Difference) strictly require **Union Compatibility**. This means: (1) both relations must possess the exact same number of attributes (identical degree), and (2) the domain of the i-th attribute in R must be identical/compatible with the domain of the i-th attribute in S.

Answer: (B)

Q7. Relation R has attributes (A, B, C) and Relation S has attributes (C, D). What is the **Degree** of the Natural Join $R \bowtie S$?

(A) 5 **(B)** 3
(C) 4 **(D)** 2

Sol. Q7. Natural Join performs an equi-join on all shared common attributes and automatically projects out duplicate column copies. The degree formula is: $\text{Degree}(R \bowtie S) = \text{Degree}(R) + \text{Degree}(S) - k$, where k is the number of shared attributes. Here, R has 3 attributes, S has 2 attributes, and they share 1 attribute (C). Degree $= 3 + 2 - 1 = 4$ (attributes: A, B, C, D).

Answer: (C)

Q8. What integrity rule states that "No component of a Primary Key in a base relation can accept a NULL value"?

(A) Referential Integrity Rule **(B)** Domain Integrity Rule
(C) User-Defined Integrity Rule **(D)** Entity Integrity Rule

Sol. Q8. The **Entity Integrity Constraint** states that no primary key attribute can be NULL because the primary key serves as the unique identifier for individual entity instances in the real world. A null value would imply an unidentified entity.

Answer: (D)

Q9. Which relational algebra operator performs a **horizontal slice** of a relation by selecting tuples that satisfy a specified predicate condition, without altering the degree of the relation?

(A) Selection (σ) **(B)** Projection (π)
(C) Join ($\bowtie$) **(D)** Cartesian Product ($\times$)

Sol. Q9. The Selection operator (σ) filters rows (tuples) based on a condition, creating a horizontal slice. It evaluates each tuple against a predicate; matching tuples are retained with all their original attributes. Hence, the degree remains unchanged.

Answer: (A)

Q10. If Relation R has 15 tuples and Relation S has 8 tuples, and the two relations are union-compatible, what is the **minimum possible Cardinality** of the Set Difference $R - S$?

(A) 0 **(B)** 7
(C) 8 **(D)** 15

Sol. Q10. Set Difference $R - S$ retains tuples that are present in R but absent in S. In the extreme case where all 8 tuples of S are also present in R (i.e. $S \subset R$), subtracting S from R removes exactly 8 tuples, leaving $15 - 8 = 7$ tuples. Hence, the minimum possible cardinality is 7.

Answer: (B)

CUET Focused Multiple Choice Questions (Part 3)

Q11. In relational database design, what is the key distinction between a **Super Key** and a **Candidate Key**?

(A) A Super Key can only contain numeric attributes, whereas a Candidate Key can contain text.
(B) A Super Key is chosen by the DBA, whereas a Candidate Key is generated by SQL.
(C) A Candidate Key is a minimal Super Key containing no redundant attributes, whereas a Super Key may contain unnecessary attributes.
(D) A Candidate Key allows NULL values, whereas a Super Key prohibits NULL values.

Sol. Q11. A Super Key is any set of attributes that uniquely identifies a tuple. A Candidate Key is a **minimal** super key: if any single attribute is removed from a candidate key, it loses its uniqueness property. Thus, all candidate keys are super keys, but super keys with extraneous attributes are not candidate keys.
Answer: (C)

Q12. Which relational algebra operator is formally utilized to answer queries that involve the universal quantifier condition **"for all"** or **"for every"** (e.g., "Find customers who bought all products")?

(A) Cartesian Product ($\times$)
(B) Equi-Join ($\bowtie$)
(C) Intersection ($\cap$)
(D) Division ($\div$)

Sol. Q12. The Division operator ($R \div S$) is specifically designed for queries involving universal quantification ("for all"). If R contains `(Customer, Product)` and S contains all `Product` IDs, $R \div S$ returns those customers who appear with every single product in S.
Answer: (D)

Q13. What is the effect on the resulting relation when the Selection operation $\sigma_{\text{Salary} > 50000}$ is evaluated on an empty relation `EMP` (having Cardinality $= 0$ and Degree $= 5$)?

(A) An empty relation with Cardinality $= 0$ and Degree $= 5$
(B) A runtime error: division by zero
(C) A relation with Cardinality $= 0$ and Degree $= 0$
(D) The operation evaluates to NULL

Sol. Q13. Evaluating a Selection condition on an empty relation produces an empty relation (Cardinality $= 0$). The schema is fully preserved, meaning the resulting relation retains all 5 original attributes (Degree $= 5$).
Answer: (A)

Q14. Consider the relations $R(A, B)$ and $S(B, C)$ where $R = \{(1, 2), (3, 4)\}$ and $S = \{(5, 6), (7, 8)\}$. What is the Cardinality of the Natural Join $R \bowtie S$?

(A) 4
(B) 0
(C) 2
(D) 1

Sol. Q14. The shared attribute is B. In R, the values of B are $\{2, 4\}$. In S, the values of B are $\{5, 7\}$. Because there are no matching values of B between R and S ($R.B \cap S.B = \varnothing$), no tuples can be joined. The result is an empty relation with Cardinality $= 0$.
Answer: (B)

Q15. Which of the following relational algebra expressions is mathematically equivalent to the Set Intersection $R \cap S$ of two union-compatible relations?

(A) $R \cup (R - S)$
(B) $(R \times S) - S$
(C) $R - (R - S)$
(D) $\pi_{R(R)} - \pi_{S(S)}$

Sol. Q15. $R - S$ contains tuples that are in R but not in S. When this set is subtracted from R, the remaining tuples are precisely those in R that **are** in S, which is the exact definition of $R \cap S$. Therefore, $R \cap S \equiv R - (R - S)$.
Answer: (C)